🌳 Tree Data Structure — Overview

A **Tree** is a **non-linear hierarchical data structure** used to represent data with a **parent-child relationship**.

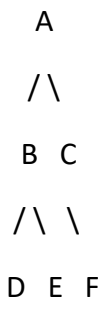
Unlike arrays, linked lists, stacks, or queues (which are **linear**), trees organize data **hierarchically**, like a **family tree or folder system** in your computer.

🧠 Basic Concepts

Let's understand key terms used in trees:

Term	Description	Example
Node	An element in the tree	A, B, C etc.
Root	The topmost node	A
Edge	Connection between nodes	(A → B)
Parent	Node with child nodes	A is parent of B, C
Child	Node derived from a parent	B and C are children of A
Leaf Node	Node with no children	E, F, G
Siblings	Nodes with same parent	B and C
Height	Longest path from root to a leaf	e.g., height = 3
Depth	Distance from root to a given node	Root has depth 0

🎄 Visual Example



- Root = A
- Children of A = B, C
- Parent of E = B
- Leaf nodes = D, E, F
- Height of tree = 3

🧩 **Types of Trees**

Type	Description	Example / Use Case
General Tree	Any node can have any number of children	XML/HTML document structure
Binary Tree	Each node has **at most 2 children**	Arithmetic expression tree
Full Binary Tree	Every node has 0 or 2 children	Used in expression parsing
Complete Binary Tree	All levels filled except possibly the last	Heap data structure
Binary Search Tree (BST)	Left child < Parent < Right child	Searching and sorting
AVL Tree	Self-balancing BST	Database indexing
Trie	Tree for storing strings	Autocomplete, Dictionary
B-Tree / B+ Tree	Used in databases and file systems	MySQL, NTFS

🧱 **Example 1 — Binary Search Tree (BST)**

A **BST** maintains the rule:

Left Subtree < Root < Right Subtree

Example:

```
    50
   /  \
```

30 70

/ \ / \

20 40 60 80

✅ **Searching for 40:**

- Start at root (50) → 40 < 50 → go left → 30 → 40 > 30 → go right → found 40.

✅ **Insertion Rule:**

- Always place smaller values on left, larger values on right.

🧬 **Tree Implementation in Python**

Example: Simple Binary Tree

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

# Create nodes
root = Node(10)
root.left = Node(5)
root.right = Node(15)
root.left.left = Node(2)
root.left.right = Node(7)

# Inorder Traversal (Left, Root, Right)
def inorder(node):
    if node:
```

```
    inorder(node.left)
    print(node.value, end=" ")
    inorder(node.right)

print("Inorder Traversal:")
inorder(root)
```

Output:

2 5 7 10 15

🔄 Tree Traversals

Traversal = visiting all nodes in a specific order.

Type	Order	Example Output (for BST above)
Inorder	Left → Root → Right	20 30 40 50 60 70 80
Preorder	Root → Left → Right	50 30 20 40 70 60 80
Postorder	Left → Right → Root	20 40 30 60 80 70 50
Level Order	Level by level	50 30 70 20 40 60 80

💡 Use Cases of Tree Data Structure

Use Case	Type of Tree Used	Explanation
File System (folders)	General Tree	Each folder has subfolders/files
Searching efficiently	Binary Search Tree	Speeds up lookup
Database Indexing	B-Tree / B+ Tree	Keeps data sorted and searchable
Expression Evaluation	Expression Tree	For parsing arithmetic expressions
Auto-complete feature	Trie	Efficient word prefix search

Use Case	Type of Tree Used	Explanation
Network Routing	Spanning Tree	Ensures no loops in network paths
Machine Learning	Decision Tree	Used in classification models

✅ **Key Advantages**

- Hierarchical data representation
- Fast searching (BST, AVL, etc.)
- Used in many real-world systems (DBs, compilers, OS)

⚠️ **Disadvantages**

- Complex implementation
- Requires more memory (due to pointers)
- Needs balancing for efficiency (in BSTs, AVL, etc.)